TD 1 – – Fiche

Guillaume THÉMÈZE – Ingénieur
Enseignant contact@themeze.fr

Plan de la séance

Durée :	🕒=15 min	—	🕒🕒=30 min	—	🕒🕒🕒=45 min	—	🕒🕒🕒🕒=60 min	—	🕒🕒🕒🕒🕒=75 min
Difficulté :	⋆=très facile	—	⋆⋆=facile	—	⋆⋆⋆=moyen	—	⋆⋆⋆⋆=difficile	→	⋆⋆⋆⋆⋆=très diffic
Priorité :	⚐=1	—	⚐⚐=2	—	⚐⚐⚐=3	—	⚐⚐⚐⚐=4	→	⚐⚐⚐⚐⚐=5
Statut :	○ facultatif	—		—		—		—	⊙ essentiel

nº	Exercice	Type	Diff.	Priorité	Durée	Statut
1	Création d'un projet sous PyCharm	Prise en main de l'outil	⋆	⚐	🕒🕒	⊙ **essentiel**
2	Recherche dichotomique récursive (`binary_search`)	Débogage + amélioration de la qualité du code	⋆⋆	⚐	🕒🕒	⊙ **essentiel**
3	Tri à bulles (`bubble_sort`) et mutation en place	Débogage + amélioration de la qualité du code	⋆	⚐	🕒🕒	⊙ **essentiel**
4	Premier script scientifique : signal sinusoïdal (mauvaises pratiques)	Premier exemple scientifique + refactoring	⋆⋆	⚐	🕒🕒🕒	⊙ **essentiel**
5	Régression linéaire bruitée et effets de bord	Débogage + amélioration de la qualité du code	⋆⋆	⚐	🕒🕒	⊙ **essentiel**
6	Somme des éléments d'une liste : mauvais argument d'appel	Débogage rapide	⋆	⚐	🕒	⊙ **essentiel**

Durée totale estimée : 140 min (dont 140 min pour les exercices essentiels ; 0 exercice facultatif).

➔ • 20 min

Exercice 1. Création d'un projet sous PyCharm

★☆☆☆☆

Prise en main de l'outil

◁ Plan de la séance ▲ Plan de la séance

Avant tout calcul scientifique, il faut un environnement de travail fonctionnel. Cet exercice fait obtenir à chaque étudiant un projet PyCharm associé à un interpréteur Python 3, contenant un fichier `main.py` qui s'exécute sans erreur. Les paquets `numpy`, `matplotlib` et `scikit-learn` devront être installés dans cet interpréteur : ils sont utilisés dès les exercices suivants.

Vocabulaire : projet, interpréteur, paquet

Prérequis

Un **projet** PyCharm est un dossier de travail. Un **interpréteur** Python est le programme qui exécute le code : il possède ses propres bibliothèques installées (`numpy`, `matplotlib`, ...). Un même poste peut avoir plusieurs interpréteurs (système, Anaconda, environnements virtuels), chacun avec son propre jeu de paquets installés indépendamment des autres.

Obtenir un environnement de travail fonctionnel

Savoir créer un projet PyCharm, lui associer un interpréteur Python 3 valide, exécuter un script et lire la sortie dans la console.

Découvrir le débogueur

Repérer les raccourcis essentiels : exécuter (Maj+F10), déboguer (Maj+F9), poser un point d'arrêt (Ctrl+F8) — utilisés dans tous les exercices suivants de ce bloc.

Sécuriser le reste de la séance

Les problèmes d'interpréteur mal configuré bloquent tout le reste de la séance : il faut circuler activement dans les 10 premières minutes et valider un interpréteur fonctionnel pour chaque étudiant avant de lancer les exercices de code.

Déroulement attendu — Point de cours

1. Lancer PyCharm puis `File > New Project`.
2. Renseigner `Location` : un dossier dédié et vide, sans espace ni accent dans le chemin, hors OneDrive ou lecteur réseau.
3. Interpréteur : choisir « Previously configured » s'il existe déjà, sinon `Add Interpreter > Add Local Interpreter > System Interpreter` et choisir un Python 3 (par ex. Anaconda 3).
4. Cocher « Create a main.py welcome script » puis `Create`.
5. Exécuter `main.py` (Maj+F10) et observer la sortie dans l'onglet `Run`.

Résultat attendu : l'interpréteur s'affiche en bas à droite de la fenêtre (« Python 3.x »), et `main.py` s'exécute avec une sortie visible.

Où PyCharm va-t-il chercher les bibliothèques ? — Point de cours

Quand le code contient `import numpy`, PyCharm cherche le paquet `numpy` uniquement dans l'interpréteur associé au projet courant — jamais dans un autre interpréteur du même poste. C'est pour cette raison qu'un même paquet peut être « installé » sur la machine et pourtant introuvable dans un projet particulier.

1. Où PyCharm va-t-il chercher les bibliothèques quand vous écrivez `import numpy` ? Quelle est la différence entre le projet PyCharm et l'interpréteur Python ?

   Coup de pouce

   Un projet est un dossier ; un interpréteur est un exécutable Python avec ses propres paquets installés à côté de lui, indépendamment du dossier de travail.

   Correction

   L'interpréteur, pas le projet, détermine les paquets disponibles.

   Projet vs interpréteur — Solution détaillée

   Le projet ne contient que le code source et sa configuration (fichiers `.idea`). Les bibliothèques Python sont installées à l'intérieur de l'interpréteur choisi (dossier `site-packages`). Deux projets peuvent donc partager le même interpréteur (et les mêmes paquets), ou au contraire utiliser des interpréteurs différents avec des jeux de paquets totalement indépendants.

2. Que se passerait-il si deux projets avaient besoin de deux versions différentes d'une même bibliothèque ?

   Coup de pouce

   Un interpréteur ne peut avoir qu'une seule version installée d'un paquet donné à la fois.

   Correction

   Il faut deux interpréteurs distincts (ex. deux environnements virtuels).

   Isolation par environnement virtuel — Solution détaillée

   Un même interpréteur ne peut contenir qu'une seule version d'un paquet donné. Pour faire cohabiter deux versions différentes, il faut créer deux environnements virtuels distincts (un par projet), chacun avec sa propre version installée : c'est précisément le rôle des `venv`, introduits en prolongement de cet exercice.

Symptôme : bandeau « No Python interpreter configured for the project ». Cause : aucun interpréteur sélectionné à la création, ou projet ouvert depuis un dossier existant. Remède : `File > Settings > Project > Python Interpreter > Add Interpreter`.

`ModuleNotFoundError: No module named 'numpy'` signifie que le paquet n'est pas installé dans l'interpréteur *sélectionné* (souvent un autre Python que celui où l'étudiant a fait `pip install`). Utiliser la fenêtre `Python Packages` de PyCharm, ou `python -m pip install numpy matplotlib scikit-learn` avec l'interpréteur du projet.

Si l'étudiant saisit un chemin déjà existant, PyCharm peut créer un sous-dossier supplémentaire : faire vérifier l'arborescence dans le panneau `Project`.

Les incidents techniques sont normaux et ne doivent pas être perçus comme un échec. Profiter de cet exercice pour montrer les raccourcis utiles : Maj+F10 (exécuter), Maj+F9 (déboguer), Ctrl+F8 (point d'arrêt), Alt+Entrée (correction rapide), Ctrl+Alt+L (reformater).

Bilan — Un environnement fiable avant tout

Un projet mal configuré (mauvais interpréteur, paquets manquants) produit des erreurs qui n'ont rien à voir avec la logique du programme : toujours vérifier l'environnement avant de chercher un bug dans le code lui-même.

Exercice 2. Recherche dichotomique récursive (`binary_search`)

➡ • 25 min ★★☆☆☆

Débogage + amélioration de la qualité du code

◁ Plan de la séance ▲ Plan de la séance

On fournit `binary_search0.py`, une recherche dichotomique récursive dans une liste triée qui renvoie un résultat *faux sans planter* : elle affiche que 10 n'est pas présent dans la liste `[2, 3, 4, 10, 40]`, alors qu'il est bien à l'indice 3. Il faut localiser le bug avec le débogueur, le corriger, puis améliorer la lisibilité du code (noms, docstring, annotations de type).

Prérequis

Sur une liste triée par ordre croissant, on compare la cible à l'élément central : si la cible est plus grande, elle se trouve dans la moitié droite ; si elle est plus petite, dans la moitié gauche. On répète récursivement jusqu'à trouver la cible, ou jusqu'à ce que la zone de recherche soit vide (indice de fin $<$ indice de début), auquel cas la valeur est absente.

Observer un symptôme (résultat faux), formuler une hypothèse sur sa cause, puis utiliser le débogueur pour la vérifier — plutôt que de modifier le code par tâtonnement.

Utiliser des noms de variables explicites, une docstring au format numpydoc et des annotations de type.

Un bug silencieux est le plus dangereux

Insister sur le fait que le programme ne plante pas et affiche pourtant un résultat faux : c'est le type de bug le plus dangereux, car rien n'attire spontanément l'attention dessus. Encourager l'écriture de cas de test avant même de corriger.

Code original fourni (`binary_search0.py`)

Point de cours

```
def binary_search(arr, low, high, x):
    if high < low:
        return -1
    mid = (low + high) // 2
    if arr[mid] == x:
        return mid
    elif arr[mid] < x:
        return binary_search(arr, low, mid - 1, x)
    else:
        return binary_search(arr, mid + 1, high, x)

sorted_list = [2, 3, 4, 10, 40]
x = 10

result = binary_search(sorted_list, 0, len(sorted_list) - 1, x)
if result != -1:
    print(f"{x} is present at index {result}")
else:
    print(f"{x} is not present in list")
```

Sortie observée : `"10 is not present in list"` — un résultat **faux**, puisque 10 est bien à l'indice 3.

Localisation du bug

Point de cours

Les deux branches récursives sont **inversées**. Si `arr[mid] < x`, la cible est dans la partie **droite** (indices `mid+1` à `high`), mais le code original cherche à gauche (`low` à `mid-1`) ; c'est l'inverse dans le `else`. Trace d'exécution sur `arr = [2, 3, 4, 10, 40]`, `x = 10` :

```
appel 1 : low=0, high=4, mid=2, arr[2]=4 < 10 -> appelle (0, 1)   [devrait être (3, 4)]
appel 2 : low=0, high=1, mid=0, arr[0]=2 < 10 -> appelle (0, -1)
appel 3 : high < low -> renvoie -1
```

Code corrigé et documenté (binary_search.py)

Point de cours

```python
def binary_search(
    sorted_values: list[int],
    lower_index: int,
    upper_index: int,
    target: int
) -> int:
    """
    Recherche un element dans une liste triee par dichotomie.

    Parameters
    ----------
    sorted_values : list[int]
        Liste d’entiers triee dans l’ordre croissant.
    lower_index : int
        Indice de debut de la zone de recherche.
    upper_index : int
        Indice de fin de la zone de recherche.
    target : int
        Valeur recherchee.

    Returns
    -------
    int
        Indice de la valeur recherchee si elle est presente,
        -1 sinon.
    """
    # Si la zone de recherche est vide, la valeur n’est pas presente.
    if upper_index < lower_index:
        return -1

    # On choisit l’indice situe au milieu de la zone de recherche.
    middle_index = (lower_index + upper_index) // 2

    # Si la valeur centrale est la cible, la recherche est terminee.
    if sorted_values[middle_index] == target:
        return middle_index

    # Si la valeur centrale est plus petite que la cible, la cible
    # se trouve necessairement dans la partie DROITE (liste triee).
    elif sorted_values[middle_index] < target:
        return binary_search(
            sorted_values, middle_index + 1, upper_index, target
        )

    # Sinon, la cible se trouve dans la partie gauche.
    else:
        return binary_search(
            sorted_values, lower_index, middle_index - 1, target
        )
```

Points forts : logique corrigée, noms explicites, docstring numpydoc, annotations de type, commentaires expliquant le raisonnement. Point à améliorer : les variables globales `sorted_values` et `target` masquent (« shadowing ») les paramètres de même nom — bon prétexte pour parler de portée des variables ; il manque aussi une garde `if __name__ == "__main__":`.

Démarche de débogage conseillée

Point de cours

1. Exécuter le code original et constater le message erroné.
2. Poser un point d'arrêt sur la ligne du calcul de `mid` (Ctrl+F8), lancer le débogueur (Maj+F9).
3. Observer `low`, `high`, `mid` et `arr[mid]` à chaque appel dans le panneau Variables.
4. Faire remarquer que `arr[mid]=4 < 10` mais que la recherche continue pourtant vers la gauche.
5. Faire formuler la règle : liste triée croissante, valeur centrale plus petite que la cible ⇒ chercher à droite.
6. Corriger, relancer, valider avec plusieurs cas de test.

1. Exécuter `binary_search0.py`. Le résultat affiché (`"10 is not present in list"`) est-il correct ? Utiliser le débogueur pour identifier précisément où le raisonnement échoue.

Coup de pouce

Posez un point d'arrêt sur la ligne de calcul de `mid` et observez `arr[mid]` comparé à `x` à chaque appel.

Correction

Les branches elif/else de la récursion sont inversées.

Diagnostic

Solution détaillée

Le résultat est faux : 10 est présent à l'indice 3. En suivant l'exécution, on constate qu'à l'appel 1, `arr[mid]=4` est plus petit que la cible `10`, ce qui devrait orienter la recherche vers la *droite* (indices 3 à 4) ; or le code appelle `binary_search(arr, low, mid-1, x)`, c'est-à-dire la partie *gauche*. Les deux branches récursives sont inverties.

2. Corriger le bug, puis vérifier le résultat sur les cas de test : cible 10 (attendu : indice 3), cible 2 (indice 0), cible 40 (indice 4), cible 5 (absente, -1), liste vide (-1), liste à un seul élément.

oup de pou

Dans la branche `arr[mid] < x`, il faut appeler `binary_search(arr, mid + 1, high, x)`.

Correction

Échanger les deux appels récursifs des branches elif et else.

Correction et validation

Solution détaillée

Après échange des deux appels récursifs, tous les cas de test suivants sont vérifiés :

Liste	Cible	Résultat attendu
`[2,3,4,10,40]`	10	3
`[2,3,4,10,40]`	2	0
`[2,3,4,10,40]`	40	4
`[2,3,4,10,40]`	5	-1
`[]`	1	-1
`[7]`	7	0

3. Pourquoi la liste doit-elle impérativement être triée pour que l'algorithme fonctionne ? Combien de comparaisons au maximum sont nécessaires pour une liste de 1 000 000 d'éléments ?

oup de pou

À chaque appel récursif, la zone de recherche est divisée par deux.

Correction

Environ log2(1 000 000) ≈ 20 comparaisons au maximum.

Solution détaillée Rôle du tri et complexité

L'algorithme décide de quel côté chercher en comparant la cible à `arr[mid]` : ce raisonnement n'est valable que si la liste est triée (sinon la cible pourrait très bien se trouver du « mauvais » côté sans que l'algorithme ne le sache). Comme la zone de recherche est divisée par 2 à chaque appel, il faut au maximum $\lceil \log_2(n) \rceil$ comparaisons, soit environ 20 pour $n = 1\,000\,000$ — une complexité logarithmique bien plus favorable qu'une recherche linéaire en $O(n)$.

Corriger le symptôme plutôt que la cause

Certains étudiants inversent la comparaison (< en >) au lieu d'inverser les appels : le code renvoie alors un résultat correct sur l'exemple donné par chance, mais le sens de la logique reste faux. Faire tester avec la valeur 2, la valeur 40 et une valeur absente comme 5 pour démasquer ce faux correctif.

D'autres trient la liste ou ajoutent des `print` un peu partout sans formuler d'hypothèse claire : reprendre avec le débogueur et, si besoin, dessiner l'arbre des appels récursifs sur papier avec les indices `low`/`high` à chaque étape.

Le bug est un classique : la structure du code est bonne, seules les branches sont inversées. Laisser les étudiants formuler l'hypothèse eux-mêmes après observation du débogueur.

Bilan Un résultat correct n'est pas toujours un résultat valide

Un programme qui ne plante pas peut quand même être faux : toujours comparer avec un résultat attendu, sur plusieurs cas de test (valeur présente en début, en fin, absente, liste vide).

➔ • 20 min

Exercice 3. Tri à bulles (`bubble_sort`) et mutation en place

★☆☆☆☆

Débogage + amélioration de la qualité du code

◁ Plan de la séance ▲ Plan de la séance

On fournit `bubble_sort0.py`, un tri à bulles qui trie en fait la liste dans l'ordre **décroissant** au lieu de croissant. Il faut identifier la cause avec le débogueur, corriger, puis discuter un effet de bord discret mais important : la fonction modifie la liste passée en argument *en place*.

À chaque passage, on parcourt la liste et on échange deux éléments adjacents s'ils ne sont pas dans le bon ordre. Après i passages, les i plus grands éléments sont correctement placés en fin de liste : on peut donc réduire la zone parcourue d'un élément à chaque passage supplémentaire.

Diagnostiquer, à l'aide du débogueur, pourquoi un tri renvoie l'ordre inverse de celui attendu.

Observer qu'une fonction Python peut modifier un objet (ici une liste) reçu en argument, sans avoir besoin de renvoyer explicitement un nouvel objet.

Préparer le terrain pour la régression linéaire

La mutation en place est un sujet transversal qui réapparaît, sous une forme plus subtile, dans l'exercice sur la régression linéaire bruitée (`td1_ex5`) : ne pas hésiter à l'amorcer ici sans trop insister sur l'efficacité algorithmique elle-même.

Code original fourni (`bubble_sort0.py`) – Point de cours

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] < arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr


unsorted_list = [64, 34, 25, 12, 22, 11, 90]

# Expected output: [11, 12, 22, 25, 34, 64, 90]
print("Sorted list:", bubble_sort(unsorted_list))
```

La condition d'échange est `arr[j] < arr[j+1]` : elle échange donc les éléments quand le premier est plus **petit**, ce qui fait remonter les plus **petits** éléments en fin de liste — un tri décroissant, contrairement au commentaire « Expected output ».

Code corrigé et documenté (`bubble_sort.py`) – Point de cours

```
def bubble_sort(values: list[int]) -> list[int]:
    """Trie une liste d'entiers dans l'ordre croissant."""
    number_of_values = len(values)

    # A chaque passage, le plus grand element encore non place
    # est deplace vers la fin de la liste.
    for pass_number in range(number_of_values):

        # La partie situee apres cet indice est deja triee :
        # elle n'a plus besoin d'etre parcourue.
        for index in range(0, number_of_values - pass_number - 1):

            # Pour un ordre CROISSANT, on echange si l'element
            # de gauche est plus GRAND que celui de droite.
            if values[index] > values[index + 1]:
                values[index], values[index + 1] = (
                    values[index + 1],
                    values[index]
                )

    return values
```

Démarche de débogage conseillée – Point de cours

1. Exécuter le code original et comparer avec le commentaire « Expected output ».
2. Poser un point d'arrêt dans la boucle interne et suivre la liste après chaque échange.
3. Demander : « quel élément remonte en fin de liste à la fin de la première passe ? » (attendu : 90, le plus grand ; dans l'original c'est 11, le plus petit).
4. Corriger le sens de la comparaison et vérifier.
5. Afficher `unsorted_list` après l'appel : la variable d'origine a elle aussi été triée (mutation en place).

1. Exécuter `bubble_sort0.py`. Le résultat correspond-il au commentaire « Expected output » ? Identifier la ligne responsable.

Coup de pouce

Suivez la liste après le premier échange dans la boucle interne : le plus petit ou le plus grand élément remonte-t-il en fin de liste ?

Correction

La condition arr[j] < arr[j+1] trie en ordre décroissant.

Diagnostic — Solution détaillée

Le résultat affiché est `[90, 64, 34, 25, 22, 12, 11]` : décroissant, alors que le commentaire annonçait un ordre croissant. La condition d'échange `arr[j] < arr[j+1]` fait remonter les plus petits éléments à la fin, ce qui produit un tri décroissant.

2. Corriger la fonction pour obtenir un tri croissant. Valider sur `[64, 34, 25, 12, 22, 11, 90]`, `[]`, `[1]`, `[3, 1, 3, 2]` et le pire cas `[5, 4, 3, 2, 1]`.

Coup de pouce

Il suffit d'inverser le sens de la comparaison, pas de trier la liste par un autre moyen.

Correction

Remplacer arr[j] < arr[j+1] par values[index] > values[index+1].

Correction et validation — Solution détaillée

Avec `values[index] > values[index+1]`, tous les cas proposés sont correctement triés en ordre croissant, y compris la liste vide et la liste à un seul élément (aucune itération de boucle n'est alors nécessaire).

3. Que vaut `unsorted_list` *après* l'appel à `bubble_sort(unsorted_list)` ? Pourquoi ?

Coup de pouce

La fonction échange les éléments avec la notation `arr[j], arr[j+1] = arr[j+1], arr[j]` : sur quel objet opère-t-elle réellement ?

Correction

La liste d'origine est triée aussi : la fonction modifie l'objet reçu en place.

Mutation en place — Solution détaillée

En Python, une liste est un objet *mutable* : le paramètre `arr` (ou `values`) référence le même objet que `unsorted_list` dans le code appelant, sans en faire une copie. Les échanges effectués à l'intérieur de la fonction modifient donc directement la liste d'origine, qui se retrouve triée après l'appel — même si l'on n'utilise pas la valeur de retour de la fonction.

Masquer le bug plutôt que le corriger — Points de vigilance pédagogiques (écueils)

Certains étudiants inversent la liste avec `.reverse()` ou `[::-1]` pour « corriger » le résultat : cela masque le symptôme sans corriger l'algorithme lui-même. Demander explicitement de corriger la comparaison dans la boucle.

Perte de stabilité avec >= — Points de vigilance pédagogiques (écueils)

Remplacer < par <= ou >= peut laisser le résultat faux, ou faire des échanges inutiles d'éléments égaux. Tester avec des doublons, par exemple `[3, 1, 3, 2]`.

Confusion sur la borne — Points de vigilance pédagogiques (écueils)

Faire dessiner l'état de la liste après chaque passe : la fin de liste est déjà triée, inutile de la reparcourir ; le « -1 » évite de sortir de la liste avec `j+1`.

Aliasing : un sujet transversal — Bilan

`bubble_sort` modifie la liste *en place* ; c'est pour cela que la fonction n'a même pas besoin de renvoyer un nouvel objet pour que l'appelant voie le résultat trié. Cette même propriété devient une source d'effets de bord non désirés dans l'exercice sur la régression linéaire (`td1_ex5`).

➔ • 35 min

Exercice 4. Premier script scientifique : signal sinusoïdal (mauvaises pratiques)

★★☆☆☆

Premier exemple scientifique + refactoring

◁ Plan de la séance ▲ Plan de la séance

On fournit `first_try0.py`, dont l'en-tête annonce lui-même : « This code shows bad programming behaviors ». Le script plante avec `NameError: name 'plt' is not defined` après avoir affiché 101 valeurs. Au-delà du bug d'import, un second défaut, purement physique celui-là, se cache dans le choix de l'échantillonnage temporel : c'est lui qui doit être identifié en priorité.

Pour reconstruire correctement un signal sinusoïdal de fréquence f à partir de ses échantillons, il faut échantillonner à une fréquence f_e telle que $f_e > 2f$. En deçà, le signal reconstruit est un artefact (repliement de spectre, ou *aliasing*) qui ne ressemble en rien au signal réel.

Lire un traceback Python et localiser précisément l'import manquant responsable de la `NameError`.

Comprendre qu'un code qui s'exécute sans erreur peut néanmoins produire un résultat physiquement absurde si l'échantillonnage n'est pas cohérent avec le signal étudié.

Avec π exact, tous les échantillons du signal original tombent (à 10^{-9} près) sur des multiples de la période : le signal vaudrait quasiment 0 partout. C'est uniquement l'approximation `3.14` qui produit une courbe non nulle d'allure plausible : deux erreurs indépendantes se combinent pour fabriquer un résultat trompeur. Utile pour montrer qu'une courbe qui « a l'air de marcher » ne prouve rien.

Code original fourni (`first_try0.py`)

Point de cours

```
import numpy as nu

def fonctionA(a, b, c):
    return a * nu.sin(2*3.14 * b * c)


if __name__ == '__main__':
    a1=20
    b1=850
    c1=nu.linspace(0,50, 101)

    r1 = fonctionA(a1, b1, c1)
    print(r1)

    plt.figure()
    plt.plot(c1, r1)
    plt.show()
```

Point de cours

`plt` est utilisé mais `import matplotlib.pyplot as plt` est absent : `NameError`. Le `print(r1)` s'exécute AVANT l'erreur, donc l'étudiant voit d'abord 101 valeurs, puis le plantage à la ligne `plt.figure()`.

Bug B2 — échantillonnage physiquement absurde

Point de cours

`c1 = linspace(0, 50, 101)` donne un pas de 0,5 s, donc une fréquence d'échantillonnage $f_e = 2$ Hz, pour un signal de 850 Hz : le critère de Nyquist ($f_e > 1700$ Hz) est massivement violé. La courbe tracée est un artefact d'aliasing, sans rapport avec une vraie sinusoïde à 850 Hz.

Bug B3 — approximation 3.14 de pi

Point de cours

L'utilisation de `3.14` au lieu de `np.pi` introduit une erreur relative d'environ 5×10^{-4} sur la phase, qui devient énorme lorsque $2\pi f t$ est grand — c'est cette approximation, combinée au bug B2, qui produit une courbe d'amplitude 20 non nulle au lieu d'un signal quasiment constant à zéro.

Code corrigé et documenté (`first_try.py`)

Point de cours

```
import numpy as np
import numpy.typing as npt
import matplotlib.pyplot as plt


def generate_sine_signal(
    amplitude: float,
    frequency: float,
    time: npt.NDArray[np.float64]
) -> npt.NDArray[np.float64]:
    """Genere un signal sinusoidal."""
    return amplitude * np.sin(2 * np.pi * frequency * time)


if __name__ == ’__main__’:
    amplitude = 20
    frequency = 850

    # Fenetre courte (10 ms) et beaucoup de points :
    # pas = 1 microseconde, soit fe = 1 MHz >> 2 x 850 Hz.
    time = np.linspace(0, 0.01, 10001)

    signal = generate_sine_signal(amplitude, frequency, time)

    print(f"Amplitude : {amplitude}")
    print(f"Frequence : {frequency} Hz")
    print(f"Nombre de points : {len(time)}")

    plt.figure()
    plt.plot(time, signal)
    plt.xlabel("Temps (s)")
    plt.ylabel("Amplitude")
    plt.title("Signal sinusoidal")
    plt.grid()
    plt.show()
```

1. Exécuter `first_try0.py`. Quelle erreur apparaît, et à quelle ligne ? Corriger ce premier problème.

oup de pou

Le message d'erreur nomme précisément le symbole non défini.

Correction

NameError : name 'plt' is not defined ; il manque l'import de matplotlib.pyplot.

Diagnostic et correctif

Solution détaillée

Le traceback indique `NameError: name ’plt’ is not defined` à la ligne `plt.figure()`. Le fichier utilise `plt` sans jamais l'importer : il faut ajouter `import matplotlib.pyplot as plt` en tête de fichier.

2. Une fois `plt` importé, le script s'exécute sans erreur. Calculer la fréquence d'échantillonnage réellement utilisée (`c1 = np.linspace(0, 50, 101)`) et la comparer au critère de Nyquist pour un signal de 850 Hz. Conclure sur la validité de la courbe affichée.

Coup de pouce

101 points sur 50 secondes donnent 100 intervalles : quel est le pas temporel, puis la fréquence d'échantillonnage associée ?

Correction

fe = 2 Hz, très inférieure à 2×850 = 1700 Hz requis : le critère de Nyquist n'est pas respecté.

Solution détaillée — Calcul et diagnostic physique

Le pas temporel est $T_e = 50/100 = 0{,}5$ s, soit $f_e = 1/T_e = 2$ Hz. Le critère de Nyquist exige $f_e > 2 \times 850 = 1700$ Hz : il est massivement violé (par un facteur supérieur à 800). La courbe affichée est donc un artefact de repliement de spectre, sans rapport avec une véritable sinusoïde à 850 Hz, même si le script ne lève plus aucune erreur Python.

3. Proposer un nouveau vecteur temps qui respecte largement le critère de Nyquist pour un signal de 850 Hz, et réécrire le script en corrigeant également le nom de fonction, les variables et l'utilisation de `np.pi` à la place de `3.14`.

Coup de pouce

Choisir une fenêtre temporelle courte (quelques millisecondes) et un grand nombre de points, plutôt qu'une longue fenêtre peu échantillonnée.

Correction

time = np.linspace(0, 0.01, 10001) donne fe = 1 MHz, largement suffisant.

Solution détaillée — Correction complète

En choisissant `time = np.linspace(0, 0.01, 10001)`, le pas devient $1\,\mu s$, soit $f_e = 1$ MHz, très largement supérieur au minimum de 1700 Hz exigé par Nyquist : la courbe affichée montre alors environ 8,5 périodes d'une véritable sinusoïde à 850 Hz sur 10 ms, conforme à l'amplitude de 20 annoncée.

Nommage de variables opaques

`fonctionA`, `a`, `b`, `c`, `a1`, `b1`, `c1`, `r1` ne permettent pas de savoir que `c` est le temps ou `b` la fréquence : faire renommer systématiquement avant même de corriger le bug physique.

Une confusion fréquente consiste à ne pas distinguer la fréquence du signal (850 Hz) de la fréquence d'échantillonnage déduite du vecteur temps : bien faire calculer explicitement $f_e = 1/T_e$ à partir du pas du vecteur temps avant toute conclusion.

Le bug d'import (B1) est trivial à corriger et masque le vrai enjeu pédagogique de l'exercice. Une fois `plt` importé, la courbe s'affiche *sans erreur Python* : c'est le moment de demander « cette courbe est-elle plausible pour un signal à 850 Hz ? » plutôt que de considérer l'exercice terminé.

Bilan — Un code sans erreur peut être physiquement faux

Un script qui s'exécute sans lever d'exception ne garantit rien sur la validité physique de son résultat. Toujours vérifier l'ordre de grandeur (ici, le nombre de points par période du signal) avant de faire confiance à un tracé.

➔ • 30 min ★★☆☆☆

Exercice 5. Régression linéaire bruitée et effets de bord

Débogage + amélioration de la qualité du code

◁ Plan de la séance ▲ Plan de la séance

On fournit `linear_regression_and_noise0.py`, qui ajoute du bruit à des données idéales, effectue une régression linéaire avec scikit-learn, puis affiche le résultat pour sept niveaux de bruit croissants. Le script ne plante pas, mais son graphique est trompeur : le bruit affiché semble s'accumuler d'un niveau à l'autre.

Prérequis

`sklearn.linear_model.LinearRegression` ajuste une droite $y = ax + b$ au sens des moindres carrés à partir de données `x` (mises en forme en colonne avec `x.reshape(-1, 1)`) et `y`. Les attributs `model.coef_[0]` et `model.intercept_` donnent respectivement la pente et l'ordonnée à l'origine estimées.

Comprendre pourquoi `y += noise` modifie un tableau `y` partagé entre plusieurs appels de fonction, et pourquoi cela fausse la comparaison entre niveaux de bruit successifs.

Ne pas confondre les données bruitées affichées et les valeurs prédites par le modèle de régression.

Un effet de bord plus subtil

Contrairement à `bubble_sort` (`td1_ex3`), où la mutation en place était le résultat recherché, ici elle est un *bug* : `y` désigne les données idéales, partagées entre tous les appels de la boucle sur les niveaux de bruit, et `y += noise` l'altère de façon cumulative et invisible.

Code original fourni (extrait)

```
def plot_linear_regression_with_noise(x, y, noise_level):
    # Add noise to the y data
    noise = np.random.normal(0, noise_level, y.shape)
    y += noise                      # <-- mutation en place de y !

    x_reshaped = x.reshape(-1, 1)
    model = LinearRegression()
    model.fit(x_reshaped, y)

    y = model.predict(x_reshaped)   # <-- y devient la PREDICTION

    slope = model.coef_[0]
    intercept = model.intercept_

    plt.figure()
    plt.scatter(x, y, color='blue', label='Noisy data ...')  # trace la prediction, pas le bru
    plt.plot(x, y, color='red', linewidth=2, label='Linear regression')
    ...

x = np.linspace(0, 10, 20)
y = 2.5 * x - 4.2

noise_levels = [0.5, 1, 2, 3, 5, 7, 10]
for noise_level in noise_levels:
    plot_linear_regression_with_noise(x, y, noise_level)
```

Deux bugs liés à la même variable y

- `y += noise` modifie `y` *en place*. Comme `y` est défini une seule fois avant la boucle et réutilisé à chaque appel, le bruit s'accumule d'un niveau de bruit au suivant : le niveau de bruit affiché au 7^{e} appel contient en réalité la somme des sept bruits successifs.
- Ensuite, `y = model.predict(x_reshaped)` **réaffecte** `y` à la prédiction du modèle : le `plt.scatter(x, y, ...)` censé afficher les « données bruitées » affiche en fait la droite de régression elle-même, rendue comme un nuage de points confondu avec la droite tracée juste après.

Code corrigé et documenté (extrait)

Point de cours

```
def plot_linear_regression_with_noise(x, y, noise_level, ax):
    """Ajoute du bruit, effectue une regression, affiche le resultat."""

    # Bruit genere a chaque appel, jamais reassigne dans y directement.
    noise = np.random.normal(0, noise_level, y.shape)
    y_noisy = y + noise             # nouvelle variable, y reste intact

    x_reshaped = x.reshape(-1, 1)
    model = LinearRegression()
    model.fit(x_reshaped, y_noisy)

    y_predicted = model.predict(x_reshaped)   # nom distinct de y_noisy

    slope = model.coef_[0]
    intercept = model.intercept_

    ax.scatter(x, y_noisy, label=f"Bruit = {noise_level}")
    ax.plot(x, y_predicted, linewidth=2, label="Regression")
    ...

np.random.seed(0)
x = np.linspace(0, 10, 20)
y = 2.5 * x - 4.2          # jamais modifie ensuite

fig, axes = plt.subplots(2, 3, figsize=(12, 7))
axes = axes.ravel()
for ax, noise_level in zip(axes, noise_levels):
    plot_linear_regression_with_noise(x, y, noise_level, ax)
```

Amélioration supplémentaire : chaque niveau de bruit est affiché sur un sous-graphique dédié (`plt.subplots(2, 3)`) plutôt que sur sept figures séparées — plus lisible pour comparer les sept cas.

1. Exécuter `linear_regression_and_noise0.py`. Comparer visuellement l'allure du nuage de points affiché pour le niveau de bruit « 0.5 » et celui affiché pour le niveau « 10 ». Que remarquez-vous d'anormal si l'on observe successivement toutes les figures produites ?

   Coup de pouce

   La variable y est-elle réinitialisée à chaque appel de la fonction, ou réutilisée telle qu'elle a été laissée par l'appel précédent ?

   Correction

   Le bruit affiché à chaque niveau contient en réalité la somme de tous les bruits précédents.

   Diagnostic de l'accumulation

   Solution détaillée

   La ligne `y += noise` modifie y en place. Comme la même variable y, définie une seule fois avant la boucle, est passée à chaque appel de `plot_linear_regression_with_noise`, chaque appel ajoute son propre bruit à un y déjà modifié par les appels précédents : le bruit s'accumule d'un niveau au suivant, au lieu de repartir des données idéales à chaque fois.

2. Le graphique légendé « Noisy data » affiche-t-il vraiment les données bruitées, ou autre chose ? Justifier en suivant la succession des affectations à la variable `y` dans la fonction.

   Coup de pouce

   Repérez toutes les lignes où y apparaît à gauche du signe = (ou de +=) à l'intérieur de la fonction.

   Correction

   y est réaffecté à model.predict(...) avant le tracé : le scatter affiche la prédiction, pas le bruit.

Traçage des affectations
Solution détaillée

Dans la fonction originale, y désigne successivement : (1) les données idéales bruitées (après `y += noise`), puis (2) les valeurs prédites par le modèle (après `y = model.predict(x_reshaped)`). Le `plt.scatter(x, y, ..., label='Noisy data ...')` est exécuté *après* cette seconde affectation : il affiche donc la prédiction du modèle, superposée avec la droite de régression elle-même tracée juste après — le nuage « bruité » légendé n'a en réalité plus rien de bruité.

3. Corriger les deux problèmes en introduisant des noms de variables distincts pour les données idéales, les données bruitées et la prédiction du modèle. Vérifier que chaque sous-figure ne contient plus que le bruit correspondant à son propre niveau.

Coup de pouce

Une seule règle suffit : ne jamais réaffecter (ni modifier en place) une variable qui doit encore servir de référence pour les appels suivants.

Correction

Introduire `y_noisy = y + noise` et `y_predicted = model.predict(...)` sans jamais modifier y.

Correction
Solution détaillée

En remplaçant `y += noise` par `y_noisy = y + noise` (nouvelle variable, y inchangé) et `y = model.predict(...)` par `y_predicted = model.predict(...)`, chaque appel repart bien des données idéales d'origine : les sept sous-figures affichent alors chacune un bruit indépendant, cohérent avec son propre niveau, et un nuage de points réellement bruité distinct de la droite de régression.

Confondre += et une nouvelle affectation
Points pédagogiques (écueils spécifiques)

`y += noise` n'est pas équivalent, pour un tableau Numpy, à `y = y + noise` du point de vue des effets de bord : les deux calculent la même valeur, mais `y += noise` modifie l'objet y existant en place (si y est un `ndarray`), alors qu'une variable `y_noisy = y + noise` crée un nouvel objet et laisse y intact.

Ne pas masquer l'accumulation du bruit
Points pédagogiques (écueils

Le script original ne plante pas et produit sept figures d'aspect plausible : l'accumulation du bruit n'est visible qu'en comparant soigneusement les niveaux affichés à l'écran (par ex. le nuage de points du niveau « 0.5 » ne doit contenir que le bruit de ce niveau, pas la somme de tous les niveaux précédents).

Isoler les données d'entrée des résultats calculés
Bilan

Ne jamais réutiliser le même nom de variable pour désigner successivement une donnée d'entrée puis un résultat calculé à partir d'elle (ici y, tour à tour données idéales, données bruitées, puis prédiction) : cela rend les effets de bord invisibles à la lecture du code.

➜ • 10 min

Exercice 6. Somme des éléments d'une liste : mauvais argument d'appel

★☆☆☆☆

Débogage rapide

◁ Plan de la séance ▲ Plan de la séance

On fournit `sum_list_elements0.py` : la fonction `sum_list_elements` elle-même est correcte, mais le script affiche `Sum of elements = 10` au lieu des `15` annoncés par le commentaire « Expected output ». Exercice très court, à traiter en échauffement ou en fin de séance.

Slicing Python
Prérequis

Pour une liste `L`, `L[:-1]` désigne tous les éléments sauf le dernier. C'est un slicing valide syntaxiquement, mais qui change le contenu réellement transmis à une fonction si on ne le fait pas intentionnellement.

Distinguer bug de logique interne et mauvais appel
Points pédagogiques (é

Vérifier qu'une fonction, correcte en elle-même, peut donner un résultat faux si elle est appelée avec de mauvais arguments.

Exercice d'échauffement ou de fin de séance

Volontairement bref : à utiliser pour occuper les étudiants les plus rapides en attendant la suite, ou pour clore la séance sur une note simple après des exercices plus longs.

Code fourni (`sum_list_elements0.py`)

Point de cours

```
def sum_list_elements(my_list):
    total = 0
    for i in range(len(my_list)):
        total += my_list[i]
    return total


sample_list = [1, 2, 3, 4, 5]

# Expected output: Sum of elements = 15
print("Sum of elements =", sum_list_elements(sample_list[:-1]))
```

Localisation du bug

La fonction `sum_list_elements` elle-même est correcte : elle additionne bien tous les éléments qu'on lui donne. Le bug est dans l'**appel** : `sample_list[:-1]` transmet `[1, 2, 3, 4]` (sans le dernier élément, 5), et non `sample_list` en entier. La somme calculée, $1 + 2 + 3 + 4 = 10$, est donc exacte *pour les arguments reçus*, mais ne correspond pas à l'intention du commentaire.

1. Exécuter le script. Le résultat correspond-il au commentaire « Expected output : Sum of elements = 15 » ? Localiser précisément la cause de l'écart, sans modifier la fonction `sum_list_elements`.

   Affichez `sample_list[:-1]` séparément avant l'appel à la fonction.

   Correction

   `sample_list[:-1]` exclut le dernier élément (5) de l'argument transmis.

   Diagnostic — Solution détaillée

   Le résultat affiché est 10, et non 15. La fonction `sum_list_elements` additionne correctement tous les éléments qu'elle reçoit, mais l'appel `sum_list_elements(sample_list[:-1])` lui transmet `[1, 2, 3, 4]` (le slicing `[:-1]` exclut le dernier élément), et non la liste complète `[1, 2, 3, 4, 5]`.

2. Corriger l'appel pour obtenir `Sum of elements = 15`.

   Il suffit de transmettre la liste complète, sans slicing.

   Correction

   `sum_list_elements(sample_list)` (sans le [:-1]).

   Correction — Solution détaillée

   En appelant `sum_list_elements(sample_list)` (sans le slicing `[:-1]`), les cinq éléments sont bien additionnés et le résultat affiché devient 15, conforme au commentaire.

Chercher le bug dans la boucle plutôt que dans l'appel

Un réflexe fréquent est de relire la boucle `for` en cherchant une erreur d'indice, alors que la boucle est parfaitement correcte : faire d'abord vérifier ce qui est réellement passé en argument, par exemple en affichant `sample_list[:-1]` avant l'appel.

Une fonction correcte peut l'être mal utilisée

Bilan

Avant de chercher un bug *à l'intérieur* d'une fonction, vérifier ce qui lui est réellement transmis en argument : le problème peut très bien se situer du côté de l'appelant.

Bilan et prolongements du TD